

Rotation-Aware Point-Cloud Embeddings for Vision-Based In-Hand Reorientation

Yashom Dighe

Karthik Dantu

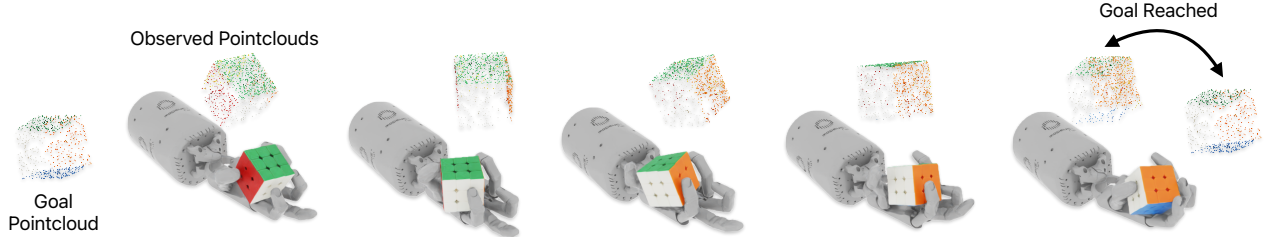


Fig. 1. Policy rollout. We train a rotation-aware point-cloud encoder that enables direct RL policy learning from observed and goal point clouds, rotating the object toward the target geometry without object pose, relative pose, dense flow, or teacher-action supervision.

Abstract—Point-cloud goals provide a direct way to specify dexterous in-hand reorientation: instead of defining an object-specific pose frame or estimating 6D pose at test time, the policy is given the desired 3D geometry of the object. Yet raw point-cloud goal conditioning is poorly conditioned for policy learning. Current and goal clouds are unordered, independently sampled, and often visibility-dependent, so their discrepancy entangles object rotation with permutation, resampling, and unstable correspondence structure. For this reason, prior point-cloud manipulation methods typically add structure outside the representation itself, such as explicit pose or relative-pose inputs, dense flow features, or distillation from privileged teachers. We close this gap by learning a rotation-aware point-cloud embedding whose Euclidean latent distance is calibrated to the $SO(3)$ geodesic error between object orientations. The resulting representation turns current-goal comparison into a smooth control signal, allowing a model-free RL policy to act from current/goal point-cloud embeddings, proprioception, and centroid metadata, without object pose, relative pose, dense flow, or teacher-action supervision. In in-hand reorientation experiments, this interface matches privileged-state and distillation-based baselines while avoiding brittle test-time computation of structured pose or flow inputs.¹

Index Terms—Learning representations for robotic perception and control, Robotic Manipulation, In-hand Manipulation

I. INTRODUCTION

Point clouds are a natural goal representation for geometric manipulation: they explicitly specify what the robot should make the scene look like in 3D. For a task like in-hand reorientation, this means the goal can be given by showing the object in its desired orientation, rather than defining an object-specific pose frame or requiring a 6D pose estimate at policy inference. Our conjecture is that since point clouds encode shape, pose, contact geometry, and spatial arrangement, a policy should in principle be able to solve such tasks by comparing the current point cloud to a goal point cloud without any additional inputs. In practice, point-cloud goal

conditioning is difficult because the current-goal comparison is hard to make control-friendly in raw point-set space. In this paper, we show that metric learning can embed point clouds in a representation where this comparison becomes feasible for dexterous control, without providing the policy object pose, relative pose, dense flow, or distillation from a privileged teacher.

The complexity of using point clouds comes from the representation itself: they are unordered surface samples with no persistent identity across observations. As a result, changes in the observation are entangled with permutation, resampling, visibility, and discontinuous nearest-neighbor assignments making it. Prior work has made point-cloud goals usable by adding structure outside the raw point-cloud representation. Geometry-Dex [1] learn RL policies that use point-cloud observations for object geometry, but supplies the policy with explicit pose information for the reorientation objective. Flow-based methods such as HACMan also learn RL policies by providing structured current-goal features by specifying how points should move from the current geometry toward the target geometry [2]. These approaches are effective but also presuppose that these structured quantities can be computed and specified robustly. While recent methods such as FoundationPose [3] have made substantial progress in 6D object pose estimation, estimating pose reliably from visual observations remains a challenging problem, as reflected by benchmarks such as BOP [4]. Similarly, robust point-cloud registration under partial observations and occlusion is still difficult; for example, HACMan [2] identifies inaccurate registration as one of its failure modes. Distillation provides another form of scaffolding: [5], [6] first train a privileged teacher and then train a point-cloud visual student to imitate its actions. This approach works around the brittle pose estimation/registration operation by directly learning on the raw pointclouds by providing stable action supervision. But, distillation can result in a learned representation that is coupled to a particular embodiment and to the exploration

¹Full Paper with appendix: <https://arxiv.org/abs/2606.21788>

or data collection policy that generated the dataset and fundamentally depends on the availability of quality expert supervision or trajectories labeled with robot actions.

Our goal is to remove this scaffolding and ask whether the representation itself can make point-cloud goal conditioning usable for standard policy learning. A generic point-cloud encoder trained for reconstruction, shape, or identity may preserve what object is present while failing to organize the task-relevant state in a way that makes current-goal comparison useful for control. We propose that, given two point clouds of the same object under different $SO(3)$ transforms, the difference in their latent representation should reflect the difference in the transforms. To achieve this, we train a point-cloud encoder with a metric-learning [7] objective that calibrates latent distance to the task’s $SO(3)$ geodesic error. We then use embeddings of the current and goal point clouds as the observation interface to solve the down-stream task of in-hand reorientation using model-free RL **without using** object pose, relative pose, dense flow, or teacher-action supervision. We demonstrate that a model-free policy not only learns stably from our representation, but also matches the performance of a distillation baseline and out-performs methods using privileged information like pose, and flow in a deployment setting. This is a demanding test because model-free RL must learn from reward rather than per-step target actions. In contrast, a policy using a task-agnostic PointMAE encoder fine-tuned on the same YCB objects fails to learn, suggesting that generic point-cloud features do not provide the task-relevant signal needed for reorientation.

II. METHODOLOGY

A. Pose-Aware Encoder

We learn a PointNet++ [8] based point-cloud encoder, ϕ , that maps each colored object cloud $\mathcal{P} \in \mathbb{R}^{N \times 6}$ to a compact embedding $z = \phi(\mathcal{P})$. The encoder is trained in a Siamese style [9] to independently encode point clouds as 128-dimensional embeddings whose Euclidean distance matches the geodesic distance between object orientations. Formally, for two clouds of the same object at orientations q_1 and q_2 , we regress the Euclidean latent distance to the $SO(3)$ geodesic rotation error,

$$\mathcal{L}_{\text{metric}} = \mathbb{E}[(\|\phi(\mathcal{P}_1) - \phi(\mathcal{P}_2)\|_2 - 2 \arccos |\langle q_1, q_2 \rangle|)^2] \quad (1)$$

Thus nearby orientations are encouraged to have nearby embeddings and distant orientations to have distant embeddings.

We train the encoder on colored point clouds (x, y, z, r, g, b) sampled from the textured mesh models in the YCB dataset [10]. A single encoder is trained across all objects rather than separately per instance. To improve robustness to partial observations that arise in the downstream in-hand reorientation task, training uses an occlusion curriculum that progressively removes points from the input clouds, exposing the model to increasingly incomplete geometry. We then fine-tune the encoder on a smaller IsaacLab dataset generated by rendering objects with RGB-D cameras and back-projecting the depth images

into point clouds, producing hand-like occlusions that better match the observations seen during policy learning.

For learning the reorientation policy, ϕ is frozen and applied separately to the current and goal clouds.

B. In-hand reorientation

The downstream task of in-hand reorientation can be formulated as a goal-augmented Markov decision process $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{G}, p, r, \rho_0, \gamma \rangle$. At the start of each episode, a goal $g \in \mathcal{G}$ is sampled from a goal distribution $p(g)$. In our setting $g \equiv q_g \in SO(3)$. The environment is initialized by sampling $(s_0, g) \sim \rho_0(\cdot)$ and the agent interacts for T steps by selecting actions $a_t \sim \pi(a_t | o_t, g)$, inducing transitions $s_{t+1} \sim p(s_{t+1} | s_t, a_t, g)$. The objective is to learn a policy that maximizes the expected discounted return over goals

$$\max_{\pi} \mathbb{E}_{g \sim p(g)} \left[\mathbb{E}_{\tau \sim \pi, p} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, a_t, g) \right] \right].$$

We use the standard IsaacLab [11] in-hand reorientation task definition, with no task-specific modifications.

System: The robot is a simulated Shadow Hand [12] with 20 actuated joints manipulating a single rigid object in-hand. The object is chosen from the YCB dataset [10]. The policy outputs normalized joint commands mapped to joint position targets.

Goals and progress metric: At the start of an episode (and optionally upon success), we sample a goal orientation $q_g \in SO(3)$. The task also defines a nominal in-hand reference position p_{ref} to which the object should remain close to encourage stable grasps and prevent drops. Progress toward the goal is measured on $SO(3)$ via the geodesic rotation distance $d_R(q, q_g) \in [0, \pi]$, implemented as the angle of the relative quaternion $q \otimes q_g^*$. An episode is considered successful when $d_R(q_t, q_g) < \beta$.

Reward: The reward encourages minimizing in-hand position drift and reducing rotational error, with action regularization and a success bonus:

$$r(s_t, a_t, g) = w_p \|p_t - p_{\text{ref}}\|_2 + w_R \frac{1}{d_R(q_t, q_g) + \epsilon} + w_a \|a_t\|_2^2 + \begin{cases} b, & \text{if } d_R(q_t, q_g) < \beta \\ 0, & \text{otherwise.} \end{cases}$$

Episodes terminate early if the object moves too far from p_{ref} (drop/out-of-reach), and otherwise time out after 600 environment steps. This count is reset upon a successful reorientation. We set the maximum number of consecutive goal successes to 5.

We solve the reorientation MDP using PPO [13] with an asymmetric actor-critic architecture.

a) *Observation Space:* The actor network receives an observation vector o_t comprising:

- **Learned embeddings:** The current object embedding $z_{\text{obj}} = \phi(\mathcal{P}_t)$ and the goal embedding $z_{\text{goal}} = \phi(\mathcal{P}_{\text{goal}})$ (128 dimensions each, 256 total).
- **Proprioception:** Joint positions, velocities, and hand-relative fingertip poses (133 dimensions).

- **Geometric metadata:** The centroids of the current and goal point clouds in the environment frame (6 dimensions total), providing explicit relative translation information to supplement the rotation-focused embeddings.

b) *Asymmetric Critic:* To facilitate faster convergence, we employ an asymmetric critic that has access to privileged state information s_t , including the ground-truth object state (pose, velocity), hand state, and physical contact forces.

III. RESULTS & EVALUATION

A. Policy Learning

First, we evaluate whether a policy using our learned point-cloud representation can match policies that use more structured object representations. We compare against baselines inspired by teacher-student distillation [5], [6] and per-point dense flow [2]. Both baselines are implemented using publicly available code.

All methods use the same IsaacLab environment with three (128×128) RGB-D views. We back-project and segment the views into a fused partial object cloud [14]; the goal cloud is sampled from the canonical mesh at the target orientation. Point jitter and shuffling augment the observations.

All policies receive proprioception and differ only in how object and goal information are represented: our method uses learned point-cloud embeddings, teacher baselines use pose information from simulator state or FoundationPose [3], the distilled student uses raw current/goal point clouds, and flow baselines use dense displacement fields computed from simulator state or registration using a standard Open3D pipeline [15].

We train separate per-object policies for four YCB objects: peach, pear, Rubik’s cube, and tuna can, using RSL-RL [16]. We report the number of successful reorientations and the average number of timesteps required in 100 independent trials. Table I presents the benchmark results. The policy trained with our representation consistently succeeds, keeping up with the distilled student, teacher, and flow baselines. However, it takes the longer to achieve success as indicated by the avg. timesteps column. During training, the first half of each run is dominated by maximizing the number of successes, after which the policy begins optimizing for speed. Our training runs are capped at 10^8 environment interaction steps. Training for longer lets the policy learn faster motions even under our observations.

Furthermore, the significant gap between the privileged and non-privileged variants of the estimator- and registration-based baselines highlights the difficulty of recovering structured intermediate quantities in a deployment setting. As stated earlier, our approach is motivated by this observation: rather than imposing pose, registration, or dense correspondence as intermediate structure, we learn a representation in which current and goal point clouds can be compared directly at the embedding level. Our policy and the distilled operate using visual input directly letting them bypass the brittle upstream modules like pose estimator and pointcloud registration.

B. Representation Diagnostics

We evaluate whether the learned embeddings preserve rotation structure. We compare our encoder with PointMAE [17], fine-tuned on the same YCB objects; the PointMAE policy failed to learn.

Using pear as a representative object, we sweep rotations about pairs of axes and measure distances between the resulting embeddings. Figure 3 shows that our encoder produces a smoother, more structured distance landscape, qualitatively matching the action-supervised student without using teacher actions.

We also test whether the embedding geometry captures rotation axes. Figure 5 shows that our encoder and the student organize same-axis rotations more consistently than PointMAE, while separating rotations around different axes. Finally, t-SNE visualizations in Figure 4 show that our embeddings separate rotations by angle and axis, except near zero rotation where the axes become indistinguishable. These results indicate that generic point-cloud features do not provide the structured signal needed for reorientation.

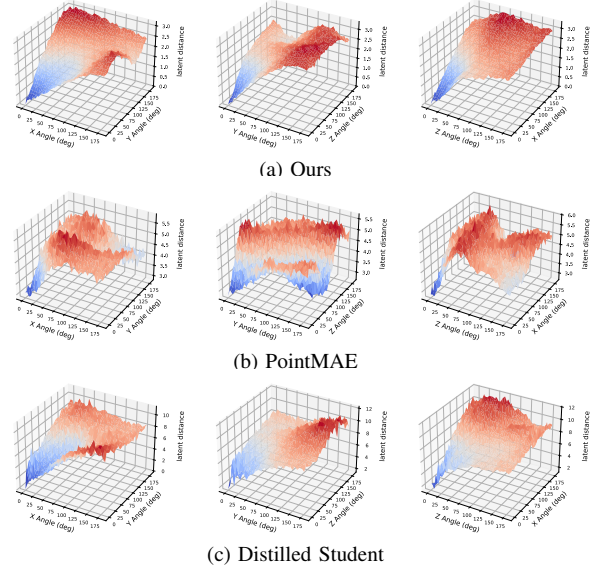


Fig. 3. Latent embedding distance as a function of simultaneous rotations around pairs of axes (X-Y, Y-Z, Z-X). Surfaces show how the encoders respond to 3D rotations of pear point clouds from 0° to 180° . Color scale: blue (low distance) to red (high distance).

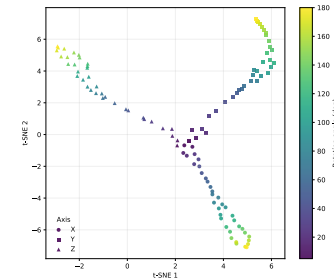


Fig. 4. Pear t-SNE.

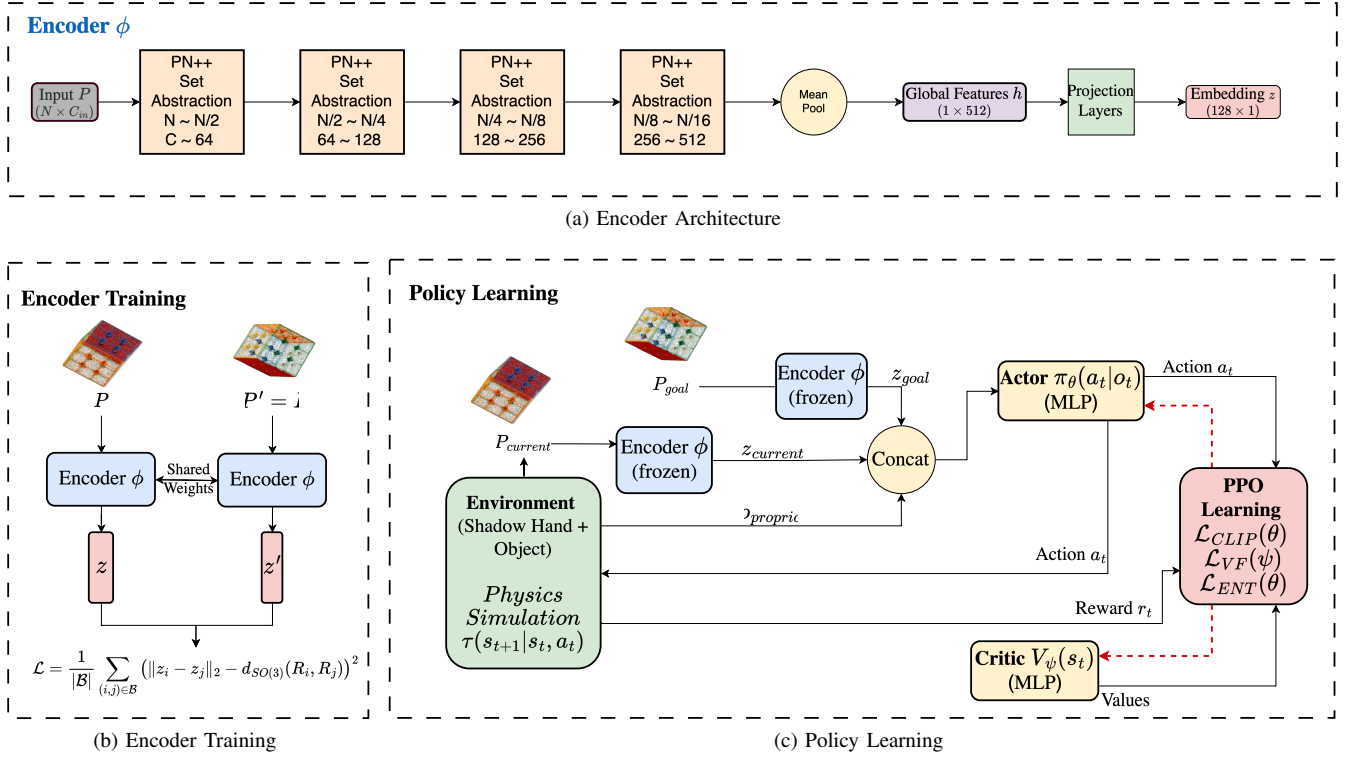


Fig. 2. Overview of the method. (a) Encoder architecture. (b) Encoder training uses paired rotations of the same object and regresses $\|z_i - z_j\|_2$ to the $SO(3)$ geodesic distance $d_{SO(3)}(R_i, R_j)$. (c) Policy learning freezes the encoder and feeds current/goal embeddings, centroids, and proprioception to an actor-critic trained with PPO ($\mathcal{L}_{CLIP}, \mathcal{L}_{VF}, \mathcal{L}_{ENT}$).

TABLE I

BENCHMARK RESULTS. NUMBER OF SUCCESSFUL REORIENTATIONS OVER 100 EVALUATION TRIALS AND AVERAGE TIMESTEPS TO REORIENT. EACH TRIAL CONSISTS OF ONE RANDOMLY SAMPLED TARGET ORIENTATION.

Object	Method											
	Ours		Teacher (sim state)		Teacher (estimator)		Distilled Student		Flow (sim state)		Flow (registration)	
	successes	Avg. timesteps	successes	Avg. timesteps	successes	Avg. timesteps	successes	Avg. timesteps	successes	Avg. timesteps	successes	Avg. timesteps
Peach	100	85.29	100	25.40	53	598.8	100	50.32	100	69.65	37	930.11
Rubik's Cube	100	71.39	100	40.21	33	1556.0	100	41.58	100	47.53	33	756.3
Pear	100	116.76	100	48.27	58	942.1	100	52.32	100	137.05	22	633.5
Tuna Can	100	154.05	100	92.36	48	1037.8	100	129.30	100	313.99	28	835.2

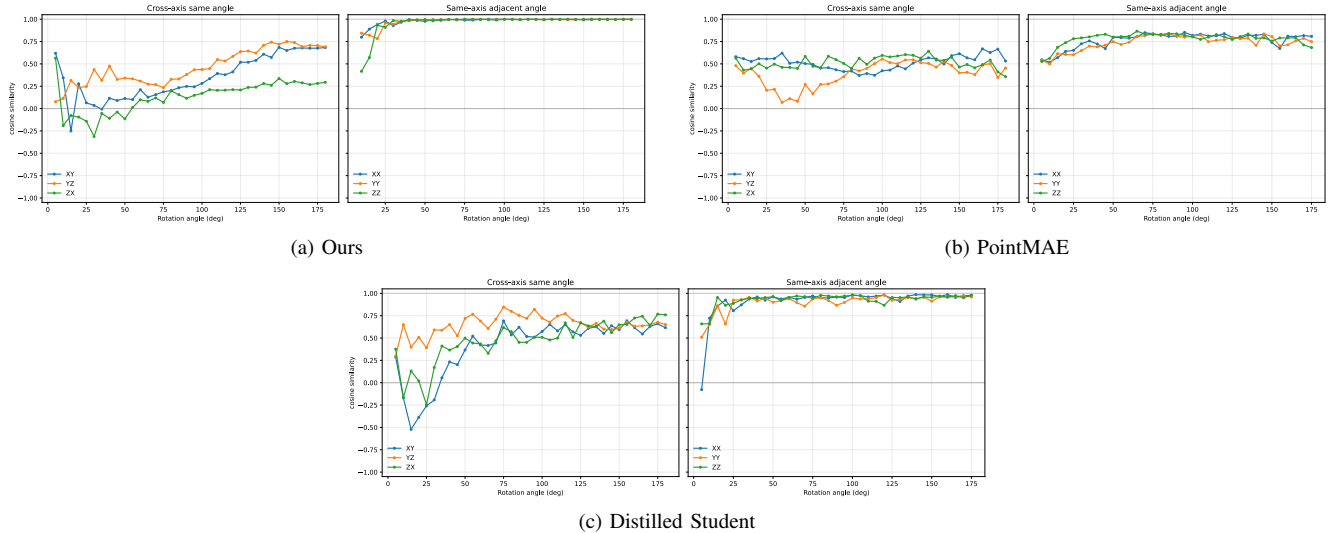


Fig. 5. Cosine similarity between embedding deltas induced by rotations around different axes for pear.

REFERENCES

- [1] W. Huang, I. Mordatch, P. Abbeel, and D. Pathak, “Generalization in dexterous manipulation via geometry-aware multi-task learning,” *arXiv preprint arXiv:2111.03062*, 2021.
- [2] W. Zhou, B. Jiang, F. Yang, C. Paxton, and D. Held, “Hacman: Learning hybrid actor-critic maps for 6d non-prehensile manipulation,” *arXiv preprint arXiv:2305.03942*, 2023.
- [3] B. Wen, W. Yang, J. Kautz, and S. Birchfield, “Foundationpose: Unified 6d pose estimation and tracking of novel objects,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.08344>
- [4] T. Hodan, F. Michel, E. Brachmann, W. Kehl, A. GlentBuch, D. Kraft, B. Drost, J. Vidal, S. Ihrke, X. Zabulis *et al.*, “Bop: Benchmark for 6d object pose estimation,” in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 19–34.
- [5] T. Chen, M. Tippur, S. Wu, V. Kumar, E. Adelson, and P. Agrawal, “Visual dexterity: In-hand reorientation of novel and complex object shapes,” *Science Robotics*, vol. 8, no. 84, p. eadc9244, 2023.
- [6] T. Chen, J. Xu, and P. Agrawal, “A system for general in-hand object re-orientation,” *Conference on Robot Learning*, 2021.
- [7] M. Kaya and H. Ş. Bilge, “Deep metric learning: A survey,” *Symmetry*, vol. 11, no. 9, p. 1066, 2019.
- [8] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “Pointnet++: Deep hierarchical feature learning on point sets in a metric space,” *Advances in neural information processing systems*, vol. 30, 2017.
- [9] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, “Signature verification using a” siamese” time delay neural network,” *Advances in neural information processing systems*, vol. 6, 1993.
- [10] B. Calli, A. Singh, J. Bruce, A. Walsman, K. Konolige, S. Srinivasa, P. Abbeel, and A. M. Dollar, “Yale-cmu-berkeley dataset for robotic manipulation research,” *The International Journal of Robotics Research*, vol. 36, no. 3, pp. 261–268, 2017.
- [11] M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Muñoz, X. Yao, R. Zurbrugg, N. Rudin, L. Wawrzyniak, M. Rakhsha, A. Denzler, E. Heiden, A. Borovicka, O. Ahmed, I. Akinola, A. Anwar, M. T. Carlson, J. Y. Feng, A. Garg, R. Gasoto, L. Gulich, Y. Guo, M. Gussert, A. Hansen, M. Kulkarni, C. Li, W. Liu, V. Makovychuk, G. Malczyk, H. Mazhar, M. Moghani, A. Murali, M. Noseworthy, A. Poddubny, N. Ratliff, W. Rehberg, C. Schwarke, R. Singh, J. L. Smith, B. Tang, R. Thaker, M. Trepte, K. V. Wyk, F. Yu, A. Millane, V. Ramasamy, R. Steiner, S. Subramanian, C. Volk, C. Chen, N. Jawale, A. V. Kuruttukulam, M. A. Lin, A. Mandlekar, K. Patzwaldt, J. Welsh, H. Zhao, F. Anes, J.-F. Lafleche, N. Moënnel-Loccoz, S. Park, R. Stepinski, D. V. Gelder, C. Amevor, J. Carius, J. Chang, A. H. Chen, P. de Heras Ciechowski, G. Daviet, M. Mohajerani, J. von Mural, V. Reutsky, M. Sauter, S. Schirm, E. L. Shi, P. Terdiman, K. Vilella, T. Widmer, G. Yeoman, T. Chen, S. Grizan, C. Li, L. Li, C. Smith, R. Wiltz, K. Alexis, Y. Chang, D. Chu, L. J. Fan, F. Farshidian, A. Handa, S. Huang, M. Hutter, Y. Narang, S. Pouya, S. Sheng, Y. Zhu, M. Macklin, A. Moravanszky, P. Reist, Y. Guo, D. Hoeller, and G. State, “Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning,” *arXiv preprint arXiv:2511.04831*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.04831>
- [12] “Shadow Dexterous Hand Series - Research and Development Tool — shadowrobot.com,” <https://www.shadowrobot.com/dexterous-hand-series/>, [Accessed 27-05-2026].
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [14] R. Szeliski, *Computer vision: algorithms and applications*. Springer Nature, 2022.
- [15] “Colored point cloud registration - Open3D primary (unknown) documentation — open3d.org,” https://www.open3d.org/docs/latest/tutorial/pipelines/colored_pointcloud_registration.html, [Accessed 28-05-2026].
- [16] C. Schwarke, M. Mittal, N. Rudin, D. Hoeller, and M. Hutter, “Rsl-rl: A learning library for robotics research,” *arXiv preprint arXiv:2509.10771*, 2025.
- [17] Y. Pang, W. Wang, F. E. Tay, W. Liu, Y. Tian, and L. Yuan, “Masked autoencoders for point cloud self-supervised learning,” in *Computer Vision—ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part II*. Springer, 2022, pp. 604–621.